

Module 1: Random Number Generation and Simulation Foundations

MA 563 – Statistical Computation and Simulation

Dr. Rishikesh Yadav

`rishikesh@iitmandi.ac.in` — TA: Vedant Vibhor (`D24028@students.iitmandi.ac.in`)

School of Mathematics and Statistical Sciences, IIT Mandi

Table of Contents

Pseudo-Random Number Generators

Motivation and Desired Properties

Linear Congruential Generators (LCG)

Modern Generators

Generating Uniform and Non-Uniform Random Variables

From PRNG Output to $U(0, 1)$

Strategies for Non-Uniform Generation

Inverse Transform Method

Theory: The Probability Integral Transform

Algorithm and Continuous Examples

The Discrete Case

Limitations

Acceptance-Rejection Method

Setup and Theory

Proof of Correctness and Efficiency

Worked Examples

Box-Muller Transformation

Derivation

Basic Algorithm

Polar (Marsaglia) Variant

Module 1: Random Number Generation and Simulation Foundations (8 Lectures)

Pseudo-random number generators, uniform and non-uniform random variable generation, inverse transform method, acceptance–rejection method, Box–Muller transformation, ratio-of-uniforms method, diagnosing randomness, and simulation error.

Module 1: Random Number Generation and Simulation Foundations (8 Lectures)

Pseudo-random number generators, uniform and non-uniform random variable generation, inverse transform method, acceptance–rejection method, Box–Muller transformation, ratio-of-uniforms method, diagnosing randomness, and simulation error.

Why start here? ✓

Every simulation-based method in this course – Monte Carlo integration, bootstrap, EM, MCMC – assumes you can already generate reliable random numbers from arbitrary distributions. Module 1 builds that foundation from first principles.

Pseudo-Random Number Generators

- Motivation and Desired Properties

- Linear Congruential Generators (LCG)

- Modern Generators

Generating Uniform and Non-Uniform Random Variables

- Inverse Transform Method

- Acceptance–Rejection Method

- Box–Muller Transformation

- Ratio-of-Uniforms Method

- Diagnosing Randomness

- Simulation Error

- Unit 6

Why “Pseudo”-Random?

- Computers are deterministic machines: given the same input, they always produce the same output.

Why “Pseudo”-Random?

- Computers are deterministic machines: given the same input, they always produce the same output.
- True randomness would need a physical source (radioactive decay, thermal noise, atmospheric noise). This is slow, non-reproducible, and impractical for billions of simulation draws.

Why “Pseudo”-Random?

- Computers are deterministic machines: given the same input, they always produce the same output.
- True randomness would need a physical source (radioactive decay, thermal noise, atmospheric noise). This is slow, non-reproducible, and impractical for billions of simulation draws.
- Instead, we use a **deterministic algorithm** that produces a sequence of numbers *statistically indistinguishable* from i.i.d. $\text{Uniform}(0, 1)$ draws.

Why “Pseudo”-Random?

- Computers are deterministic machines: given the same input, they always produce the same output.
- True randomness would need a physical source (radioactive decay, thermal noise, atmospheric noise). This is slow, non-reproducible, and impractical for billions of simulation draws.
- Instead, we use a **deterministic algorithm** that produces a sequence of numbers *statistically indistinguishable* from i.i.d. $\text{Uniform}(0, 1)$ draws.

Why “Pseudo”-Random?

- Computers are deterministic machines: given the same input, they always produce the same output.
- True randomness would need a physical source (radioactive decay, thermal noise, atmospheric noise). This is slow, non-reproducible, and impractical for billions of simulation draws.
- Instead, we use a **deterministic algorithm** that produces a sequence of numbers *statistically indistinguishable* from i.i.d. $\text{Uniform}(0, 1)$ draws.

Definition: Pseudo-Random Number Generator (PRNG) ✓

A PRNG is a deterministic algorithm that, starting from an initial value called the seed X_0 , produces a sequence $X_1, X_2, X_3, \dots$ that mimics the statistical behaviour of i.i.d. uniform random variables.

Desired Properties of a Good PRNG

- **Uniformity:** the output values should be spread evenly over $(0, 1)$.

$$x \in \mathcal{U}(0, 1)$$

Desired Properties of a Good PRNG

- **Uniformity:** the output values should be spread evenly over $(0, 1)$.
- **Independence:** successive values should show no detectable correlation or pattern.

Desired Properties of a Good PRNG

- **Uniformity:** the output values should be spread evenly over $(0, 1)$.
- **Independence:** successive values should show no detectable correlation or pattern.
- **Long period:** the sequence should not repeat for an extremely long time (period $\gg$ number of draws needed).

Desired Properties of a Good PRNG

- **Uniformity:** the output values should be spread evenly over $(0, 1)$.
- **Independence:** successive values should show no detectable correlation or pattern.
- **Long period:** the sequence should not repeat for an extremely long time (period $\gg$ number of draws needed).
- **Reproducibility:** the same seed must always regenerate the exact same sequence (essential for debugging and replicable research).

$$x_1 \dots x_n \rightarrow \frac{x_0}{\text{current time}}$$

Desired Properties of a Good PRNG

- **Uniformity:** the output values should be spread evenly over $(0, 1)$.
- **Independence:** successive values should show no detectable correlation or pattern.
- **Long period:** the sequence should not repeat for an extremely long time (period $\gg$ number of draws needed).
- **Reproducibility:** the same seed must always regenerate the exact same sequence (essential for debugging and replicable research).
- **Efficiency:** generation must be fast – simulations often require billions of draws.

Desired Properties of a Good PRNG

- ✓ • **Uniformity:** the output values should be spread evenly over $(0, 1)$.
- ✓ • **Independence:** successive values should show no detectable correlation or pattern.
- ✓ • **Long period:** the sequence should not repeat for an extremely long time (period $\gg$ number of draws needed).
- ✓ • **Reproducibility:** the same seed must always regenerate the exact same sequence (essential for debugging and replicable research).
- ✓ • **Efficiency:** generation must be fast – simulations often require billions of draws.
- ✓ • **Portability:** the same seed should give the same sequence across machines/languages when required.

The Linear Congruential Generator (LCG)

Recurrence

$$X_{n+1} = (aX_n + c) \bmod m, \quad U_n = \frac{X_n}{m}$$

$x_0, a, c, m \Rightarrow (a, c, m)$

$x_1 = (ax_0 + c) \bmod m$

$x_2 = (ax_1 + c) \bmod m$

$x_3 = (ax_2 + c) \bmod m$

$\vdots$

$m = 16$

$0, 1, \dots, 15, 0, 1, \dots$

$m = 2^{32}$

The Linear Congruential Generator (LCG)

Recurrence

$$X_{n+1} = (aX_n + c) \bmod m,$$

$$U_n = \frac{X_n}{m}$$

Handwritten notes: A red arrow points from the fraction to the text $0 \leq U_n < 1$. The denominator m is circled in red.

- m – the **modulus** ($m > 0$)
- a – the **multiplier** ($0 \leq a < m$) ✓
- c – the **increment** ($0 \leq c < m$) ✓
- X_0 – the **seed** ($0 \leq X_0 < m$) ✓

The Linear Congruential Generator (LCG)

Recurrence

$$X_{n+1} = (aX_n + c) \bmod m, \quad U_n = \frac{X_n}{m}$$

- m – the **modulus** ($m > 0$)
- a – the **multiplier** ($0 \leq a < m$)
- c – the **increment** ($0 \leq c < m$)
- X_0 – the **seed** ($0 \leq X_0 < m$)
- If $c = 0$: called a **multiplicative congruential generator (MCG)**.
- If $c \neq 0$: called a **mixed congruential generator**.
- The sequence $\{X_n\}$ is purely deterministic once (a, c, m, X_0) are fixed – but $\{U_n\}$ can pass strong statistical tests for uniformity and independence.

Full-Period Conditions: The Hull–Dobell Theorem

Theorem (Hull–Dobell, 1962)

An LCG has full period m (i.e., it cycles through all integers $0, \dots, m - 1$ before repeating) if and only if:

1. $\gcd(c, m) = 1$ (c and m are coprime);
2. $a - 1$ is divisible by every prime factor of m ;
3. $a - 1$ is divisible by 4, if m is divisible by 4.

$m = 10^{24}$

Full-Period Conditions: The Hull–Dobell Theorem

Theorem (Hull–Dobell, 1962)

An LCG has full period m (i.e., it cycles through all integers $0, \dots, m - 1$ before repeating) if and only if:

1. $\gcd(c, m) = 1$ (c and m are coprime);
2. $a - 1$ is divisible by every prime factor of m ;
3. $a - 1$ is divisible by 4, if m is divisible by 4.

Classic Example: Lehmer / Park–Miller Generator

$m = 2^{31} - 1 = 2147483647$ (a Mersenne prime), $a = 7^5 = 16807$, $c = 0$

This multiplicative generator has period $m - 1$ and was, for decades, the default generator in many statistical packages.

$$x_{n+1} = (ax_n + c) \bmod m$$

$$x_0 \rightarrow \begin{pmatrix} m = 2^{31} - 1 \\ a = 7^5 \\ c = 0 \end{pmatrix}$$

Worked Numerical Example

Take a tiny toy LCG: $m = 16$, $a = 5$, $c = 3$, $X_0 = 7$.

LCG →

$$X_{n+1} = (5X_n + 3) \bmod 16$$

→ LCG

	x_0	x_1	x_2	x_3	x_4	x_5	$x_6 \dots$	x_{16}
n	0	1	2	3	4	5		
X_n	7	6	1	8	11	10		↓ 7, 6
$U_n = X_n/16$	0.4375	0.375	0.0625	0.500	0.6875	0.625		

- Check Hull–Dobell: $\gcd(3, 16) = 1$ ✓; prime factor of 16 is 2, and $a - 1 = 4$ divisible by 2 ✓; m divisible by 4 and $a - 1 = 4$ divisible by 4 ✓ $\Rightarrow$ full period 16.

Worked Numerical Example

Take a tiny toy LCG: $m = 16$, $a = 5$, $c = 3$, $X_0 = 7$.

$$X_{n+1} = (5X_n + 3) \bmod 16$$

n	0	1	2	3	4	5
X_n	7	6	1	8	11	10
$U_n = X_n/16$	0.4375	0.375	0.0625	0.500	0.6875	0.625

- Check Hull–Dobell: $\gcd(3, 16) = 1$ ✓; prime factor of 16 is 2, and $a - 1 = 4$ divisible by 2 ✓; m divisible by 4 and $a - 1 = 4$ divisible by 4 ✓ $\Rightarrow$ full period 16.
- In practice m is astronomically large (e.g. $2^{31} - 1$ or bigger), so the period is enormous.